Five-Bar Linkage — STM32H750 Firmware Setup Plan (UART / ODrive ASCII)

Replaces the laptop-side NiceGUI dashboard's motion path with firmware running on a custom STM32H750VBT6 PCB. The MCU initializes the ODrive, then continuously listens for `(Y, timestamp)` packets over USB CDC from a laptop, runs the same inverse-kinematics math as the dashboard (with X held fixed), and streams joint commands to the ODrive over UART using its ASCII protocol.

---

1. System architecture

```
[Laptop]
   |  USB CDC  (packet: header | Y | timestamp | checksum)
   ▼
[STM32H750 firmware]
   ├─ USB CDC RX → ring buffer → packet parser (state machine)
   ├─ Boot / init state machine (relay → NRST → ODrive UART bring-up)
   ├─ IK module (ported from NiceGUI dashboard, X fixed)
   ├─ deg → turns conversion (per-axis gear/offset/direction/home)
   ├─ ODrive UART ASCII driver (position writes + periodic error polls)
   └─ Watchdog / fault handling
        |  UART (ODrive UART_A)
        ▼
    [ODrive] → Motor A (axis0) / Motor B (axis1)
```

---

2. CubeMX peripheral setup

Peripheral	Purpose	Notes
USB_OTG (FS or HS), device mode, CDC class	Virtual COM port to laptop for coordinate packets	
USART (to ODrive UART_A)	ODrive ASCII protocol link	Baud rate must match `odrv0.config.uart_a_baudrate` on the ODrive (commonly 115200 — confirm your ODrive config)

Peripheral	Purpose	Notes
DMA (RX, circular) + USART IDLE line interrupt	Robust variable-length ASCII line reception from ODrive	Standard pattern; avoids per-byte ISR overhead and dropped bytes mid-line
GPIO: `Relay_EN`	Power relay for ODrive/motor stage	Output push-pull
GPIO: `ODrive_NRST`	ODrive reset line	Output push-pull
TIM (free-running, µs or ms tick)	Packet staleness checks, poll timing, non-blocking delays	Optional but recommended
IWDG	Recover from a hung parser/UART bus	Strongly recommended for anything driving motors

3. Boot / init state machine

```c
typedef enum {
    BOOT_POWER_UP,
    BOOT_RELAY_ON,
    BOOT_ODRIVE_RESET,
    BOOT_WAIT_ODRIVE_READY,
    BOOT_SET_CONTROL_MODE,
    BOOT_REQUEST_CLOSED_LOOP,
    BOOT_RUNNING,
    BOOT_FAULT
} boot_state_t;

static boot_state_t boot_state = BOOT_POWER_UP;
```

State transitions

`BOOT_POWER_UP`

- `HAL_Delay(6000);` — rail settle time, as given.

- → `BOOT_RELAY_ON`

`BOOT_RELAY_ON`

- `HAL_GPIO_WritePin(Relay_EN_GPIO_Port, Relay_EN_Pin, GPIO_PIN_SET);`
- `HAL_Delay(4000);`
- → `BOOT_ODRIVE_RESET`

`BOOT_ODRIVE_RESET`

- `HAL_GPIO_WritePin(ODrive_NRST_GPIO_Port, ODrive_NRST_Pin, GPIO_PIN_RESET);`
- `HAL_Delay(1000);`
- `HAL_GPIO_WritePin(ODrive_NRST_GPIO_Port, ODrive_NRST_Pin, GPIO_PIN_SET);`
- → `BOOT_WAIT_ODRIVE_READY`

`BOOT_WAIT_ODRIVE_READY`

- Every ~200 ms, send `r axis0.current_state\n` over UART.
- On a valid numeric reply (any value, just confirms the ODrive UART is alive and answering) → `BOOT_SET_CONTROL_MODE`.
- Timeout after N retries (e.g. 25 × 200 ms = 5 s) → `BOOT_FAULT`.

`BOOT_SET_CONTROL_MODE`

- Send (fire-and-forget, `w` writes don't reply):

```
w axis0.controller.config.control_mode 3\n   // POSITION_CONTROL
w axis0.controller.config.input_mode 1\n     // PASSTHROUGH
w axis1.controller.config.control_mode 3\n
w axis1.controller.config.input_mode 1\n
```

- → `BOOT_REQUEST_CLOSED_LOOP`

`BOOT_REQUEST_CLOSED_LOOP`

- Send:

```
w axis0.requested_state 8\n   // AXIS_STATE_CLOSED_LOOP_CONTROL
w axis1.requested_state 8\n
```

- Poll `r axis0.current_state\n` / `r axis1.current_state\n` until both read `8`.

- Poll `r axis0.error\n` / `r axis1.error\n` until both read `0`.
- On success → `BOOT_RUNNING`.
- On error code present, or timeout (e.g. 5 s) → `BOOT_FAULT`.

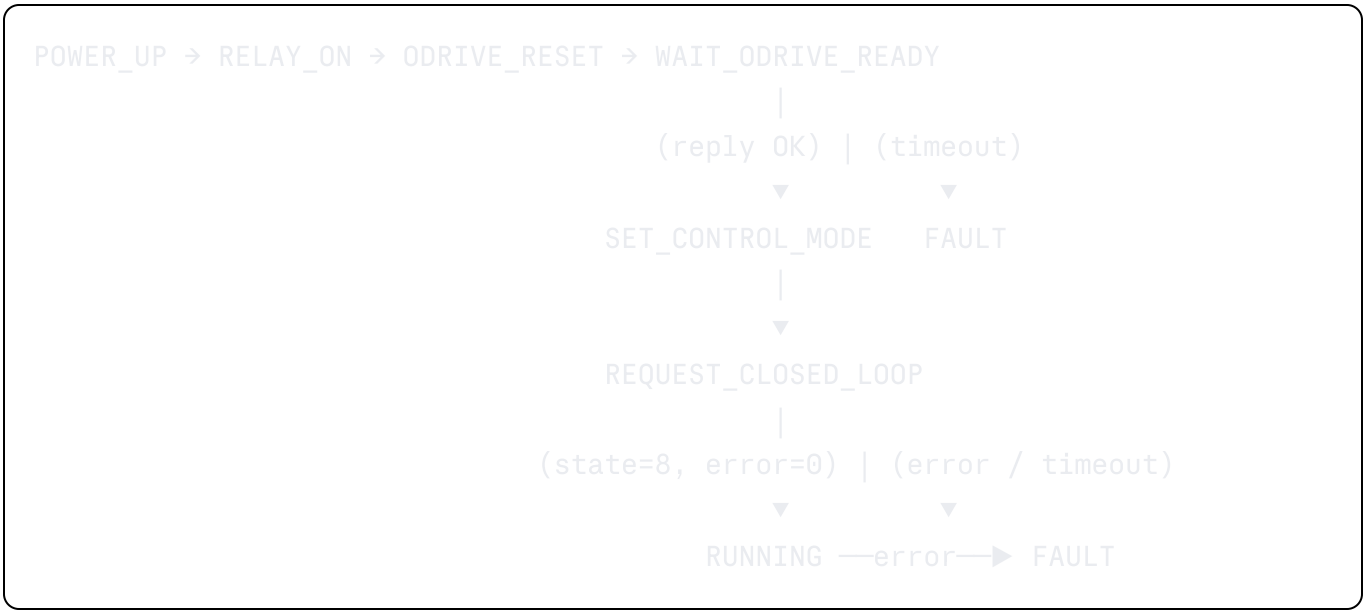
`BOOT_RUNNING`

- Normal operation: parse incoming USB CDC packets, run IK, send `p` commands.
- Periodically (rate-limited, e.g. every 100–200 ms) poll `r axis0.error\n` / `r axis1.error\n`. Any non-zero → `BOOT_FAULT`.

`BOOT_FAULT`

- Stop issuing `p` position commands (gate all sends on `boot_state == BOOT_RUNNING`).
- Optionally: drop `Relay_EN`, blink a status LED, and require a manual/external reset or re-attempt of `BOOT_ODRIVE_RESET` after a cooldown.

State diagram



---

4. USB CDC packet protocol

X is fixed in firmware (pick-line constant), so the incoming packet only needs a header, Y, and a timestamp, plus a checksum for corruption detection:

```
c
```

```c
#pragma pack(push, 1)
typedef struct {
    uint8_t  header[2];     // sync bytes, e.g. 0xAA 0x55
    float    y_mm;          // target Y coordinate
    uint32_t timestamp_ms; // sender-side timestamp
    uint8_t  checksum;     // e.g. XOR over y_mm + timestamp_ms bytes
} coord_packet_t;
#pragma pack(pop)
```

Reception:

- `CDC_Receive_FS` callback pushes received bytes into a ring buffer only — do not parse inside the callback/ISR.
- Main loop scans the ring buffer for the `0xAA 0x55` sync sequence, then reads the fixed remaining length, verifies the checksum, and resyncs on mismatch (discard one byte and rescan).

Staleness / ordering guard:

- Track `last_accepted_timestamp_ms`. Reject (silently drop) any packet whose `timestamp_ms` is older than the last accepted one, or older than `now - STALE_THRESHOLD_MS`. This prevents a paused/backlogged sender from flooding stale targets once it resumes.

---

5. Inverse kinematics (ported from dashboard, X fixed)

```c
```

```c
#define FIXED_X_MM   (0.0f)   // set to your actual pick-line X

typedef struct {
    float L0, l1a, l2a, l1b, l2b;
    int8_t elbow1, elbow2;    // +1 = "up", -1 = "down"  (mirrors Python elbow
} ik_params_t;

static float solve_arm_angle_rad(float ax, float ay, float tx, float ty,
                                 float l1, float l2, int8_t elbow_sign)
{
    float dx = tx - ax, dy = ty - ay;
    float d  = sqrtf(dx * dx + dy * dy);

    if (d > (l1 + l2) || d < fabsf(l1 - l2) || d < 1e-6f) {
        return NAN;   // unreachable — mirrors the Python ValueError
    }

    float base_angle = atan2f(dy, dx);
    float cos_val = (l1 * l1 + d * d - l2 * l2) / (2.0f * l1 * d);
    if (cos_val > 1.0f)  cos_val = 1.0f;
    if (cos_val < -1.0f) cos_val = -1.0f;
    float elbow_angle = acosf(cos_val);

    return base_angle + elbow_sign * elbow_angle;
}

void inverse_kinematics(float y, const ik_params_t *p,
                        float *theta1_deg, float *theta2_deg)
{
    float ax = -p->L0 / 2.0f, ay = 0.0f;   // Motor A (axis0)
    float bx =  p->L0 / 2.0f, by = 0.0f;   // Motor B (axis1)

    float t1 = solve_arm_angle_rad(ax, ay, FIXED_X_MM, y, p->l1a, p->l2a, p->el
    float t2 = solve_arm_angle_rad(bx, by, FIXED_X_MM, y, p->l1b, p->l2b, p->el

    *theta1_deg = t1 * (180.0f / (float)M_PI);
    *theta2_deg = t2 * (180.0f / (float)M_PI);
}
```

Always check `isnan()` on both outputs before proceeding — an unreachable target must be rejected (logged as a fault / packet skipped), never sent to the ODrive.

6. Joint-degree → ODrive-turns conversion

Direct port of the dashboard's `joint_deg_to_turns`:

```c
typedef struct {
    float gear_ratio;
    float offset_turns;
    float direction;   // +1.0 or -1.0
} axis_cfg_t;

float joint_deg_to_turns(float angle_deg, const axis_cfg_t *cfg, float home_ang
{
    float normalized = angle_deg - home_angle_deg;
    return cfg->offset_turns + cfg->direction * (normalized / 360.0f) * cfg->ge
}
```

`home_angle_deg[0]` / `home_angle_deg[1]` correspond to the dashboard's per-axis "Sync Now (Current Pose = 90°)" calibration. Decide up front:

- **Bake in** as constants once you've calibrated on the bench, or
- Implement a firmware-side re-zero trigger (spare GPIO button, or a dedicated packet type) if the mechanism will need re-homing without a reflash.

`L0`, `l1a`, `l2a`, `l1b`, `l2b`, `elbow1`, `elbow2` (link geometry) should similarly start as `#define`s and move into your existing flash-backed config module once validated on hardware.

---

7. ODrive UART ASCII driver

Position command format:

```
p <axis> <pos> <vel_ff> <torque_ff>\n
```

`pos` is in turns (already the output of `joint_deg_to_turns`); feedforwards can be `0 0`.

```c
```

```c
void odrive_send_input_pos(UART_HandleTypeDef *huart, uint8_t axis, float turns
{
    char buf[64];
    int len = snprintf(buf, sizeof(buf), "p %u %.5f 0 0\n", axis, turns);
    HAL_UART_Transmit(huart, (uint8_t *)buf, len, 10);
}
```

- **Fire-and-forget:** `p`/`w` writes get no reply. Only `r` (read) commands reply, so the steady-state motion path needs no response parsing — only the boot-time and periodic health-check `r` polls do.
- **Blocking vs DMA/IT TX:** a ~20-byte line at 115200 baud is ~1.7 ms blocking — fine for bring-up. Once both axes are being written back-to-back on every incoming packet, switch to DMA or interrupt-driven TX with a small queue so the main loop never stalls on UART-busy.
- Gate every `odrive_send_input_pos()` call on `boot_state == BOOT_RUNNING`.

Response line parser (for the `r` polls in the boot state machine and the periodic error check): read bytes via the DMA+IDLE-line RX handler into a line buffer until `\n`, then `atoi`/`atof` the token before it.

---

8. Main loop

```c
```

```
while (1)
{
    boot_state_machine_step();

    if (boot_state == BOOT_RUNNING && packet_ready_in_ring_buffer())
    {
        coord_packet_t pkt;
        if (parse_and_validate_packet(&pkt))
        {
            float t1, t2;
            inverse_kinematics(pkt.y_mm, &ik_params, &t1, &t2);

            if (!isnan(t1) && !isnan(t2))
            {
                float turns0 = joint_deg_to_turns(t1, &axis_cfg[0], home_angle_
                float turns1 = joint_deg_to_turns(t2, &axis_cfg[1], home_angle_

                odrive_send_input_pos(&huart_odrive, 0, turns0);
                odrive_send_input_pos(&huart_odrive, 1, turns1);
            }
            else
            {
                log_fault(FAULT_UNREACHABLE_TARGET);
            }
        }
    }

    poll_odrive_errors_if_due();   // rate-limited, e.g. every 100-200 ms
    HAL_IWDG_Refresh(&hiwdg);
}
```

`poll_odrive_errors_if_due()` must be interval-gated (`HAL_GetTick()`-based) — it shares the same UART line as motion commands, so polling too often competes with position writes for bandwidth. Since UART `p`/`w` writes are one-way, this periodic `r axis.error` poll is your **only** real-time signal that something has gone wrong on the ODrive side (no unsolicited heartbeat like CANSimple provides), so keep the interval tight — around 100 ms is reasonable.

---

9. Open decisions before implementation

- **Command smoothing:** the dashboard streams a software trapezoidal profile before

writing `input_pos`. If the laptop sends raw Y targets at a high/uneven rate, either add equivalent smoothing in firmware or rely on `INPUT_MODE_PASSTHROUGH` + the ODrive's own velocity/accel limits (simpler, matches the given control-mode config).

- `FIXED_X_MM` **value**: confirm the actual pick-line X coordinate to bake in.
- **Homing strategy**: bake-in vs. firmware-side re-zero trigger (Section 6).
- **Timestamp usage**: staleness rejection only (recommended, simple) vs. also using it for velocity feedforward (more work, likely unnecessary for a first pass).
- **UART baud rate**: must match `odrv0.config.uart_a_baudrate` set on the ODrive itself — verify before bring-up.

---

10. Suggested file layout

```
Core/Inc/
  boot_state_machine.h
  coord_packet.h
  ik.h
  odrive_uart.h
Core/Src/
  boot_state_machine.c
  coord_packet.c
  ik.c
  odrive_uart.c
  main.c            // ties it together: init, main loop, callbacks
```